

****Question 1: Pointer Basics and Dereferencing****

Declare an integer variable `score` and initialize it to `100`. Declare an integer pointer `ptrScore` and make it point to `score`.

1. Print the value of `score` using its variable name.
2. Print the value of `score` again, but this time using `ptrScore` (dereferencing).
3. Modify the value of `score` to `95` using `ptrScore`.
4. Print the new value of `score` using both its variable name and `ptrScore`.

****Question 2: Pointers and Arrays****

Create an integer array `numbers` of size 5, initialized with values `10, 20, 30, 40, 50`. Declare an integer pointer `ptrNumbers` and make it point to the first element of the array.

Write a C++ program segment that uses `ptrNumbers` to iterate through the entire array and print each element. Ensure you are only using pointer arithmetic (e.g., `ptrNumbers++`) and dereferencing (`*ptrNumbers`) to access array elements, not array indexing (`numbers[i]`).

****Question 3: Pointer Arithmetic****

Consider the following C++ code snippet:

```
```cpp
int values[] = {100, 200, 300, 400, 500};
int* p = values; // p points to values[0]
```
```

1. What will be the output of `*(p + 2)`? Explain your reasoning.
2. What will be the output of `*(++p)` if `p` initially points to `values[0]`? Explain your reasoning.
3. Write a single line of C++ code using pointer `p` (assuming it has been reset to `values[0]`) to print the value `400`.

****Question 4: Pointers with Functions (Pass by Pointer)****

Write a C++ function named `doubleValue` that takes an integer pointer as an argument. Inside the function, double the value that the pointer points to.

In your `main` function:

1. Declare an integer variable `myNum` and initialize it to `7`.
2. Call the `doubleValue` function with `myNum` (passed as a pointer).
3. After the function call, print the value of `myNum` in `main`.
4. Explain why using a pointer was necessary here if the goal was to modify `myNum` directly within the function.

****Question 5: NULL Pointers and Error Identification****

Analyze the following C++ code snippet:

```
```cpp
#include <iostream>
```

```
int main() {
 int* ptr = nullptr;
 // Assume some complex logic where ptr might sometimes get a valid address,
 // but for this specific execution path, it remains nullptr.

 // ... more code ...

 std::cout << *ptr << std::endl; // Line X

 return 0;
}
...
```

1. What is `nullptr` used for in C++?
2. What will happen when `Line X` is executed? Explain the specific type of error or behavior you would expect, and why.
3. How can you modify the code snippet above (specifically before `Line X`) to prevent this error by safely checking the pointer before dereferencing?